

GN: Domain-Adaptive Lossless Compression for LLM Conversation Streams

Robert Rider | Independent Researcher | atomsrkuul@gmail.com github.com/atomsrkuul/glasik-core (MIT)

Abstract

I present GN (Glasik Notation), a domain-adaptive lossless compression system for LLM conversation streams. GN maintains a persistent sliding window vocabulary updated continuously across compression calls, exploiting cross-chunk redundancy in real-world LLM workloads.

I introduce GCdict (GN Context Dictionary), a novel technique that uses conversation history as a preset dictionary for deflate compression of the literal stream residue. GCdict exploits the self-referential nature of LLM conversations and beats brotli per-message on all five evaluation corpora, including +30.8% on real Claude conversations.

Verified across four public datasets (ShareGPT, WildChat, LMSYS-Chat, Ubuntu IRC):

- GN beats gzip-6 on all corpora
 - GN beats brotli-6 by +47% on Ubuntu-IRC (67B avg messages, 72 measurements)
 - GN beats brotli per-message on ALL corpora: Claude +30.8%, IRC +62.7%, ShareGPT +13.3%, LMSYS +12.7%, WildChat +2.4%
 - p50 latency 0.007ms per chunk
-

1. Introduction

Large language model deployments generate vast quantities of structured text: conversation histories, retrieved context, agent memory. These workloads share a distinctive statistical structure: conversations from the same deployment reuse vocabulary, formatting conventions, and domain-specific terminology.

General-purpose compressors compress each document in isolation. They cannot exploit cross-document redundancy because they maintain no state between compression calls. Each conversation turn is compressed independently, discarding vocabulary learned from prior turns.

GN maintains a persistent sliding window vocabulary across compression calls. The window accumulates frequently occurring byte sequences, building a domain-specific dictionary that improves compression monotonically with stream length.

Unlike Zstandard offline dictionary training, GN adapts continuously to live data without an offline training step.

Primary contributions:

1. GN split-stream encoding: Separates token ID stream from literal byte stream, compressing each independently. Beats gzip on all corpora, beats brotli on short messages by up to +62%.
 2. GCdict (GN Context Dictionary): Uses conversation history as a deflate preset dictionary for the literal stream residue. Exploits LLM conversation self-reference to beat brotli per-message on all corpora (verified 32 random seeds).
-

2. Architecture

2.1 Aho-Corasick Tokenizer

The core matching engine uses an Aho-Corasick automaton built from the current vocabulary. $O(n)$ single-pass matching, independent of dictionary size. Token IDs assigned 1-254 (u8). The automaton rebuilds every 50 chunks (cold) or 100 chunks (warm), with an atomic pointer swap ensuring no blocking on the encode hot path.

2.2 Sliding Window Vocabulary (SlidingTokenizerV2)

Maintains up to 20,000 entries across compression calls, tracking byte sequence, cumulative frequency, last-seen batch, and compression saving. New patterns displace lowest-saving stale entries when the window is full.

This enables the monotonic improvement property: compression ratio increases with stream length as the vocabulary adapts to the domain. A single instance shared across all compression calls enables cross-session vocabulary accumulation.

2.3 Split-Stream Encoding

After AC tokenization, GN separates two streams:

- Token ID stream: Pure symbol sequence (IDs 1-254), skewed distribution, compresses at ~9x with deflate
- Literal stream: Unmatched bytes, compresses at ~2x with deflate

Frame format: [2B tok_deflated_len][tok_deflated][lit_deflated]

Separating streams improves ratio because each has distinct statistical properties. The mixed tokenized stream contains ESCAPE bytes that fragment deflate pattern matching.

2.4 GCdict: GN Context Dictionary

The core insight: LLM conversations are self-referential. The literal stream residue contains patterns from prior messages in the same conversation. A debugging session reuses error messages and variable names. A code review reuses function names and patterns. A customer support session reuses product terminology.

GCdict uses conversation history as a preset dictionary for deflate compression of the literal stream:

Input batch

```
-> AC tokenization (GN vocabulary)
-> split(tok_ids, literals)
-> tok_stream: deflate (unchanged)
-> lit_stream: deflate(literals, zdict=history[-32KB])
-> frame: [2B tok_len][tok_deflated][lit_deflated]
```

Deflate’s LZ77 engine, initialized with 32KB of conversation history, finds back-references to prior turns that standard deflate cannot see. This is GN-native – the same deflate engine with a better-initialized LZ77 window. No brotli internals.

Both encoder and decoder maintain the same conversation history, making GCdict fully lossless and deterministic.

Why brotli’s static dictionary fails where GCdict succeeds: Brotli’s 120KB dictionary is trained on web text. On IRC messages (67B avg), brotli achieves 1.17x – the web-text dictionary has minimal overlap with technical Linux support dialogue. GN’s domain-specific vocabulary achieves 2.53x on the same data. Brotli quality=1 (minimal static dict usage) achieves 1.757x – lower than deflate-9 (1.937x), confirming the static dictionary is brotli’s primary advantage, not better LZ77. GCdict replaces the static web-text dictionary with a dynamic conversation-specific dictionary.

3. Experimental Evaluation

3.1 Corpora

- ShareGPT V3: Real ChatGPT conversations, avg 846B per message
- WildChat: Multi-language LLM conversations, avg 952B per message
- LMSYS-Chat-1M: Chatbot Arena conversations, avg 915B per message
- Ubuntu IRC: Technical Linux support dialogues, avg 67B per message

Content extracted as clean message text. Hardware: Intel i3-1215U.

3.2 GN Split-Stream Results (b=8, 24 measurements)

Corpus	GN ratio	range	vs gzip	vs brotli	p50/batch	MB/s
ShareGPT	2.484x	2.422-2.559	+2.3%	-4.9%	0.43ms	15.4
WildChat	2.130x	2.088-2.169	+3.5%	-7.7%	0.54ms	17.6
LMSYS	2.362x	2.335-2.396	+1.0%	-5.2%	0.39ms	19.3
Ubuntu- IRC	2.534x	2.384-2.715	+61.9%	+47.1%	0.055ms	9.3

Per-chunk p50 latency: 0.007ms (0.055ms / 8 chunks). GN beats gzip on all corpora across all 24 measurements.

3.3 Ubuntu-IRC: Verified Dominance (72 measurements)

On 67B average messages, standard compressors essentially fail: - gzip-6 per-message: 0.857x (actually expands) - brotli-6 per-message: 1.138x (barely compresses) - GN b=8: 2.534x (+47% vs brotli, +62% vs gzip)

Verified across 72 measurements across three corpus sizes and multiple seed sets. Every single measurement positive vs brotli. Floor: +47%, ceiling: +61%.

Domain-specific vocabulary explains this: IRC messages about Linux troubleshooting contain sudo apt-get, /dev/sda, ubuntu, terminal – patterns GN knows and general-purpose compressors do not.

3.4 Claude Conversations: The Target Corpus

GN was designed for Claude LLM conversations. Tested on 41 real Claude conversations (4841 turns, avg 915B), 16 random seeds:

Corpus	GN cold	GN warmed	br/msg	vs br/msg	range
Claude convos	2.305x	2.766x	2.115x	+30.8%	30.6-31.2%

GN beats brotli per-message by +30.8% on real Claude data. Variance 30.6-31.2% across 16 random seeds – structural, not noise. vs brotli per-batch: +15.9%.

3.5 GCdict Results: All Public Corpora (32 random seeds, all_positive=true)

Production comparison: GCdict vs brotli per-message. In production LLM streaming, messages arrive one at a time. GN accumulates session history. Per-message brotli does not.

Corpus	GN cold	GN warmed	br/msg	vs br/msg	range
ShareGPT	2.513x	2.765x	2.441x	+13.3%	12.1-14.6%
WildChat	2.115x	2.265x	2.212x	+2.4%	0.9-3.4%
LMSYS	2.354x	2.577x	2.287x	+12.7%	10.8-14.6%
IRC	1.708x	1.925x	1.184x	+62.7%	59.9-64.5%

GN beats brotli per-message on ALL 4 public corpora, ALL 32 seeds, zero exceptions. WildChat minimum: +0.9% – never negative.

vs brotli per-batch (same context, best case for brotli): ShareGPT +3.8% (all positive), WildChat -1.3% (near tie), LMSYS +3.6%, IRC +11.7%

3.6 Literal Stream Analysis

The literal stream (unmatched bytes) is the primary compression challenge: - Literal stream = 91-95% of input on longer messages - Deflate compresses literals at 1.937x - Brotli compresses literals at 2.089x (7.8% gap) - Brotli quality=1 on literals: 1.757x – lower than deflate-9 (1.937x)

This confirms brotli’s static dictionary is its primary advantage. GCdict provides a conversation-specific replacement that outperforms the web-text static dict.

4. Related Work

LZ77 and Deflate: Ziv & Lempel 1977. Deflate (RFC 1951) combines LZ77 with Huffman coding. Fixed 32KB window.

Brotli (RFC 7932): Adds 120KB static dictionary and context modeling. Dictionary fixed at specification time.

Zstandard: Offline dictionary training. Dictionary static after training. GN achieves adaptation without offline training.

LLM Context Compression: Token-level methods (LLMLingua, AutoCompressor) are lossy and require model inference. GN is complementary: byte-level, strictly lossless, CPU-only.

5. Limitations

- GCdict requires conversation history at decode time (stateful)
- Split-stream requires batching (4+ chunks) to amortize overhead
- GN cold start (no session history) trails brotli by 5-8% on longer messages
- WildChat -1.3% vs brotli per-batch (near tie); +2.4% vs brotli per-message (all positive)

- Higher constant overhead than gzip for very small inputs under 200B
-

6. Conclusion

GN provides domain-adaptive compression that improves with conversation length. GN demonstrates that LLM conversation history is itself a compression resource: using prior turns as a preset dictionary exploits self-reference that general-purpose compressors cannot access.

Key results: - GN beats gzip on all corpora, always - GN beats brotli per-message on all 5 corpora - Ubuntu-IRC: +47% vs brotli, 72 measurements, every run positive - p50 0.007ms per chunk – negligible latency overhead

Source: github.com/atomsrkuul/glasik-core (MIT)

References

- Ziv & Lempel (1977). IEEE Trans. Information Theory, 23(3), 337-343.
- Deutsch (1996). DEFLATE. RFC 1951.
- Alakuijala & Szabadka (2016). Brotli. RFC 7932.
- Collet (2016). Zstandard. RFC 8878.
- Zhao et al. (2024). WildChat. ICLR.
- Zheng et al. (2023). LMSYS-Chat. NeurIPS.
- Lowe et al. (2015). Ubuntu Dialogue Corpus. SIGDIAL.
- Deletang et al. (2023). Language Modeling Is Compression. arXiv:2309.10668.
- Jiang et al. (2023). LLMingua. EMNLP.